一、模板层

二、教学目标

1、掌握FBV和CBV

2、掌握过滤器的基本使用

3、掌握自定义过滤器/标签/以及inclusion_tag

三、FBV和CBV

3.1 概述

FBV:·基于函数的视图,在 views.py文件中通过创建函数来完成对用户请求的处理.

- 目前我们现在一直使用的就是FBV

CBV:基于对象的视图,视图可以是类,将视图函数的逻辑拆成类下的对个函数执行..

CBV步骤:

1 在app01应用->views.py中 创建类和方法

```
from django.views import View

#登录类
class Login(View):

    def get(self,request):
        print('提交了get请求')
        return render(request,'login.html')

    def post(self,request):
        print('提交了post请求')
        return HttpResponse('post请求')
```

2 在app01应用-> urls.py 创建路由

```
#通过 views.Login 是一个实例化类
#as_view 类视图（views.Login）本身是一个类，无法直接被调用。
#因此需要 as_view() 方法将类转换为一个符合要求的函数。
path('login/',views.Login.as_view())
```

3 在templates 创建login.html

```
<form action="" method="post">
    <p>用户名:<input type="text" name="username"></p>
    <p>密码:<input type="password" name="password"></p>
    <input type="submit">

</form>
```

3.2 完成测试

地址栏输入 http://127.0.0.1:8000/app01/login/

用户名:

密码:

提交

这里走的其实是get请求 当你提交的时候走的就是post请求

怎么做到的呢?

归功于 as_view() 这个函数一定是使用了classmethod或者staticmethod装修的函数、不然不可能可以直接可以使用(.)出来

`as_view()` 的底层逻辑

- `as_view()` **是一个类方法**，定义在 Django 的 `View` 基类中。
- 它的核心作用是生成一个

  闭包函数

  这个闭包函数会在每次请求到达时：

  1. **实例化类**：创建 `views.Login` 类的实例。
  2. **调用 `dispatch()` 方法**：根据请求的 HTTP 方法（GET、POST 等），分发到类中对应的处理函数（如 `get()`、`post()`）。

通过源码分析

```
        if not hasattr(cls, key):
            raise TypeError("%s() received an invalid keyword %r. as_view "
                            "only accepts arguments that are already "
                            "attributes of the class." % (cls.__name__, key))

    def view(request, *args, **kwargs):
        self = cls(**initkwargs)
        self.setup(request, *args, **kwargs)
        if not hasattr(self, 'request'):
            raise AttributeError(
                "%s instance has no 'request' attribute. Did you override "
                "setup() and forget to call super()?" % cls.__name__
            )
        return self.dispatch(request, *args, **kwargs)
    view.view_class = cls
    view.view_initkwargs = initkwargs
```

最终返回的函数是 dispatch

```
def dispatch(self, request, *args, **kwargs):
    # Try to dispatch to the right method; if a method doesn't exist,
    # defer to the error handler. Also defer to the error handler if the
    # request method isn't on the approved list.
    if request.method.lower() in self.http_method_names:
        handler = getattr(self, request.method.lower(), self.http_method_not_allowed)
    else:
        handler = self.http_method_not_allowed
    return handler(request, *args, **kwargs)
```

- lower() 转换小写
- names: get/post
- getattr : 映射函数,通过字符串去调用类中的方法
- 最终就是 getattr(Login(),'get') 调用Login类的get方法

四、模版层

4.1 概述

通过模版语法可以将后端的数据传给前端页面使用

语法:

- {{ }} : 变量语法 用于使用变量值
- {% %} : 语句语法 用于执行语句

变量的使用

模版语法支持传递python所有的基本数据

类型

传递函数名时,会自动执行函数获取到返回值,但是函数不支持参数

传递类名或者类的对象得到的都是对象,类也不可以有参数

传递变量名时,会自动判断当前变量是否可以调用,如果可以就执行

神奇的点. 在前端页面调用对象的属性或者方法,或者字典通过健获取值,或者列表通过索引获取元素都可以用点实现

4.2 代码实现

1 templates中创建index.html

```
<body>
<p>整数类型：{{ n }}</p>
<p>浮点数类型：{{ f }}</p>
<p>字符串类型：{{ s }}</p>
<p>布尔类型：{{ b }}</p>
<p>元组类型：{{ tuple1 }}</p>
<p>列表类型：{{ list1 }}</p>
<p>字典类型：{{ dict1 }}</p>
<p>集合类型：{{ set1 }}</p>
<p>函数：{{ func }}</p>
<p>类中函数：{{ Myclass }}</p>
<p>类中对象：{{ obj }}</p>
<p>类中对象调用方法：{{ obj.get_static }}</p>
<p>获取字典中值为列表中的第一个元素：{{ dict1.hobby.0 }}</p>
<p>获取字典中值为列表中的字典的元素：{{ dict1.hobby.2.game }}</p>
</body>
```

2 app01 应用中 url.py

```
#演示模版层中使用到的数据类型怎么返回到前端去展示
path('index/',views.index)
```

3 app01 应用中 views.py

```
#演示数据返回
def index(request):
    n = 10,# int类型
    f = 11.11,# float类型
    s = 'hello world',# 字符串类型
    b = True,# 布尔类型
    tuple1 = (1,2,3,4,5) # 元组类型
    list1 = [1,2,3,4,5,6,7] # 列表类型
    dict1 = {'username':'fengxi','age':20,'hobby':['sing','dance',{'game':'篮
球'}]} # 字典类型
    set1 = {1,2,3,4,5}# 集合类型
    #普通函数
```

```
    def func():
        print("nihao")
        return '你好'
    #类
    class Myclass:
        #对象绑定方法 通过类对象调用
        #实例方法：必须传入 self（指向实例对象）
        def get_self(self):
            return 'self'
        #类方法：必须用 @classmethod 装饰器，第一个参数是 cls（指向类本身）
        @classmethod
        def get_cls(cls):
            return 'cls'
        #静态方法：必须用 @staticmethod 装饰器，无默认参数（无需 self 或 cls）
        @staticmethod
        def get_static():
            return 'static'
    #实例化对象
    obj = Myclass
    return render(request,'index.html',locals())
```

4.3 过滤器

概述:

最多有两个参数 类似于pthon里的内置函数

过滤器用于转换变量和标签参数的值，其语法结构是：

```
{{ variable|filter:parameter }}
```

其中：

- `variable`指要修饰的变量或者标签参数变量
- `filter`指过滤器，如`add`、`lower`等
- `parameter`指过滤器的参数（可选）

4.3.1 • `length`

`length`过滤器用于返回字符或列表的长度，语法结构为：

```
{{ variable|length }}
```

代码演示:

```
<p>统计字符串长度:{{ s|length }}</p>
```

4.3.2 • `default`

`default` 过滤器用于返回默认值，语法结构为：

```
{{ variable|default }}
```

代码演示:

```
<p>默认值:{{ b|default:'默认值' }}解释: 当b有值时(True)返回b 当b没有值时(False)返回
default后面的值</p>
```

4.3.3 • `date`

`date` 过滤器用于返回表示日期格式化，语法结构为：

```
{{ variable|data:'Y-m-d h:i:s' }}
```

代码演示:

```
views.py
#传递一个日期
import datetime

time = datetime.datetime(2000, 1, 1, 8, 0, 0)

index.html
<p>格式化日期:{{ time|date:'Y-m-d h:i:s' }} 解释:Y是四位年份，m是两位月份，d是两位日期，h
是12小时制的小时，i是分钟，s是秒 time</p>
```

4.3.4 • `slice`

`slice` 过滤器返回列表的一个片段，其语法结构为：

```
{{ variable|slice:'[start]:[end]' }}
```

代码演示:

```
<p>切片:{{ list1|slice:'0:5:2' }}</p>
```

4.3.5 • `truncatechars/truncatewords`

`truncatechars`标签将用于截断指定长度的字符串，语法结构为：

```
{{ variable|truncatechars:length }}
```

代码演示:

```
views.py
  #传递一串字符串 用于截取
  chi = '小说是一种叙事性的文学体裁，通过人物的塑造和情节、环境的描述来概括地表现社会生活。“小说”一词出自《庄子·外物》。'
  us = '''Novels are a narrative literary genre that summarizes social life through the portrayal of characters and the description of plot and environment. The term 'novel' comes from Zhuangzi's' Outer Things'.'''


<p>截取中文(9个包含...)：{{ chi|truncatechars:9 }}</p>
<p>截取英文(9个不包含...)：{{ us|truncatewords:9 }}</p>
```

4.3.6 • `cut`

`cut`对字符串进行移除。其语法结构是：

```
{{ variable|cut:' ' }}
```

代码演示:

```
<p>移除字符串：{{ us|cut:' ' }}</p>
```

4.3.7 • `join`

`join`过滤器用于使用指定分隔符连接一个列表，语法结构为：

```
{{ variable|join:'separator' }}
```

代码演示:

```
<p>拼接：{{ list1|join:'-' }}</p>
```

4.3.8 • `safe`

`safe` 用于标记一个字符串在输出前不需要进一步的 `HTML` 转义，语法结构为：

```
{{ variable|safe }}
```

代码演示:

```
views.py
 #自定义一个html代码
 html_h1='<h1>你好</h1>'

index.html
<p>转义：{{ html_h1|safe }}</p>
```

4.3.9 • `floatformat`

`floatformat` 对浮点数进行四舍五入。其语法结构是：

```
{{ variable|floatformat:length }}
```

说明：如果省略 `length` 参数的话，则默认保留到整数位

代码演示:

```
<p>四舍五入：{{ f|floatformat:2 }}</p>
```

4.3.10 • `add`

`add` 标签用于将参数添加到值中，语法结构为：

```
{{ variable|add:parameter}}
```

`add` 过滤器将首先尝试将两个值强制转为整数。如果失败，则尝试将两个值拼接在一起。

代码演示:

```
views.py
str1 = "Hello"  # 字符串1
str2 = "World"  # 字符串2


index.html
<p>字符串拼接：{{ str1|add:str2 }}</p>
```

4.3.11 • `lower`

`lower`过滤器用于将字符串转换为小写字母，语法结构为：

```
{{ variable|lower }}
```

代码演示:

```
views.py

s = "hello WORLD"  # 混合大小写字符串
<!-- 小写转换 -->
<p>小写：{{ s|lower }}</p>
```

4.3.12 • `upper`

`upper`过滤器用于将字符串转换为大写字母，语法结构为：

```
{{ variable|upper }}
```

代码演示:

```

<!-- 大写转换 -->
<p>大写：{{ s|upper }}</p>
```

完成测试

统计字符串长度：11

默认值：True解释: 当b有值时(True)返回b 当b没有值时(False)返回default后面的值

格式化日期：2000-01-01 08:00:00 解释:Y是四位年份，m是两位月份，d是两位日期，h是12小时制的小时，i是分钟，s是秒 time

切片：[1, 3, 5]

截取中文(9个包含...)：小说是一种叙事性…

截取英文(9个不包含...)：Novels are a narrative literary genre that summarizes social …

移除字符串：Novelsareanarrativeliterarygenrethatsummarizessociallifethroughtheportrayalofcharactersandthedescriptionofplotandenvironment.Theterm'novel'comesfromZhuangzi's'OuterThings'.

拼接：1-2-3-4-5-6-7

转义：

你好

四舍五入：11.99

字符串拼接：HelloWorld

小写：hello world

大写：HELLO WORLD

4.4 标签(语句)

django的模版语法提供了很多标签

例如：

load,url,for,if

语法：

- for循环(按住tab健补全)

```
{% for i in list1 %}
   循环代码
{% endfor %}
```

- if判断(写if然后按tab键补齐)

```
{% if 条件1 %}
   执行代码
{% elif 条件2 %}
   执行代码
{% else %}
   执行代码
{% endif %}
```

代码演示

```
<h1>循环的演示</h1>
{% for i in list1 %}
    <p>{{ i }}</p>
    <p>{{ forloop }}</p>
{% endfor %}
<h1>if的演示</h1>
{% if b %}
    <p>hello</p>
{% elif s %}
    <p>world</p>
{% else %}
    <p>hello,world</p>
{% endif %}
```

- **forloop变量**
    - parentloop：外层循环的信息，用于嵌套循环。
    - counter0：当前循环的索引，从0开始。
    - counter：当前循环的次数，从1开始。
    - revcounter：距离循环结束还剩多少项，从1开始。
    - revcounter0：同上，从0开始。
    - first：是否第一次循环。
    - last：是否最后一次循环。

【应用场景】：可以通过forloop.first判断它是否是第一行

可以通过forloop.last判断它是否是最后一行

for循环和if判断的嵌套的使用

```
<h1>if和for混合使用的演示</h1>

{% for i in list1 %}
    {% if forloop.first %}
        <p>这是第一次循环</p>
    {% elif forloop.last %}
        <p>这是最后一次循环</p>
    {% else %}
        <p>{{ i }}</p>
    {% endif %}
{% endfor %}
```

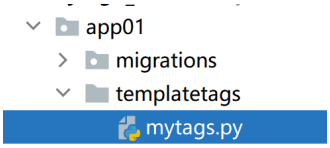
4.5 自定义过滤器/标签/inclusion_tag

上面我们使用了系统自带的一些过滤器和标签其实我们可以自定义

4.5.1 自定义过滤器

1 在应用(app01)文件下创建一个名称必须为templatetags的文件夹

2 在templatetags的文件夹中创建自定义的过滤器的名字 例如 mytags




```
from django import template
#注册一个自定义标签
register = template.Library()

#接下来就可以开始自定义过滤器,标签或者inclusion_tag
#注册自定义过滤器,名称叫 my_sum(最多可以传2个参数)
@register.filter(name='my_sum')
def my_sum(x,y):
    return x+y
```

index.html

```
<h1>自定义过滤器的使用</h1>
{% load mytags %}
<p>{{ n|my_sum:66 }}</p>
```

4.5.2 自定义标签

参数可以有多个

```
#自定义标签(参数可以有多个)
@register.simple_tag(name='yyy')#name就是自定义的标签的名称
def test(a,b,c,d):
    return '%s-%s-%s-%s' %(a,b,c,d)
```

index.html

```
<h1>自定义标签的使用</h1>
<p>{% yyy 2000 01 01  %}</p>
```

4.5.3 自定义inclusion_tag

`inclusion_tag` 是一种 Django 模板标签，允许你通过**渲染另一个模板片段**来生成动态内容。它的核心用途是**封装可复用的 UI 组件**（如导航栏、分页控件等），并在多个模板中调用。

```
# 自定义 inclusion_tag
@register.inclusion_tag('nav.html', name='show_nav')
def generate_nav(active_page='home'):
    # 定义导航栏数据
    nav_items = [
        {'name': '首页', 'url': '/', 'id': 'home'},
        {'name': '文章', 'url': '/', 'id': 'articles'},
        {'name': '关于', 'url': '/', 'id': 'about'},
    ]
    # 标记当前活跃页面
    for item in nav_items:
        item['active'] = (item['id'] == active_page)
    return {'nav_items': nav_items}  # 返回模板上下文
```

在templates中创建nav.html

```

<nav class="navbar">
    <ul>
        {% for item in nav_items %}
            <li class="{% if item.active %}active{% endif %}">
                <a href="{{ item.url }}">{{ item.name }}</a>
            </li>
        {% endfor %}
    </ul>
</nav>
```

测试

```
<h1>自定义inclusion_tag的使用</h1>
<!--'articles' 是传递给 show_nav 标签的参数，它表示当前页面对应的导航项 ID -->
{% show_nav 'articles' %}
```

五、作业

5.1 完成本天的案例